

PyTrackingAnalysis — Scripts & the Script Editor

Scripts are saved, re-runnable analysis recipes. Instead of clicking the same sequence of Hub buttons for every experiment, you record the sequence once — as an ordered list of *steps* — and run it with one click, or automatically across every replicate of a Project.

This guide covers everything about scripts: where they live, how to author them in the visual Script Editor, what every action does, how they run, and Project-level scripting (`project.yaml`).

Table of contents

1. Concepts
 2. Opening the Script Editor
 3. The editor, pane by pane
 4. Building your first script
 5. Action reference
 6. Faceting: how `facet` and `cutoffs` work
 7. Running scripts
 8. Project scripts (two-level scripting)
 9. The YAML behind it all
 10. Validation and troubleshooting
-

1. Concepts

- A **script** is a named, ordered list of steps: `{name: "...", steps: [...]}`.
 - A **step** is one action plus its parameters: `{action: run_analysis, params: {facet: true}}`.
 - Scripts exist at **two levels** (see `docs/adr/0006`):
 - **Experiment Scripts** use experiment-level actions (load, QC, analysis, plots, report) and are stored in an experiment's `tracking_config.yaml` under `scripts:` — or, for the replicates of a Project, centrally in the Project's `project.yaml` under `experiment_scripts:` (one recipe for every replicate, never copied around).
 - **Project Scripts** use project-level actions (validate design, run in experiments, project report, publication figures, AI narrative) and live in `project.yaml` under `scripts:`. See §8.
 - A file can hold **any number of scripts**, each with its own name.
 - Steps execute **top to bottom**. State flows through the run: the `load_experiment` step sets the current experiment, and every later step operates on it. Filters permanently modify the in-memory experiment for the remainder of the run (they never touch the data on disk).
-

2. Opening the Script Editor

There are two routes into the same editor:

- **Experiment scripts** — open the experiment's `tracking_config.yaml` in the Config Editor (`pytrack-config` directly, or **Experiment configs...** in the Hub's Project panel → **Experiments**

card), then click the **scripts icon** in its top bar. (The Hub's **Edit config...** button is the *Project* editor — it opens `project.yaml`, not a replicate's config.)

- **Project scripts** — click **Edit scripts...** in the Hub's Project panel → **Analysis** card. The editor opens on `project.yaml` with a **level switcher** in the top bar: *Project scripts* shows the project-action palette; *Experiment scripts* shows the familiar experiment palette editing the centrally-held `experiment_scripts`: recipes.

The editor opens as its own window, titled with the YAML file it edits. It is non-modal, so you can keep the Config Editor and Hub open alongside it.

The editor reads the YAML's `global.tracking_type` to decide which plot actions to offer, and `global.facet_cutoffs` to show inherited cutoff defaults — so open it on the same YAML your experiment uses.

3. The editor, pane by pane

Palette (actions)	Canvas (ordered steps)	Inspector (parameters)
YAML preview		[Reload] [Save]

Top bar — script switcher. A drop-down of every script in the file, plus three small buttons: **new** (create a named empty script), **rename**, and **delete** (with confirmation). The path bar below shows the YAML file being edited and an amber • **unsaved changes** indicator when there are unsaved edits.

Palette (left). Every available action as a tile, grouped by category (Load, QC, Analyze, Plots). Plot actions are filtered to the project's `tracking_type` — e.g. the *PI* tile only appears for two-choice types. **Double-click a tile** to append it as a step to the current script.

Canvas (center). The script itself: one card per step, in execution order. Each card shows the action's icon and title, a short summary of its parameters, and buttons to **move up / move down / delete**. Click a card to select it and edit its parameters in the Inspector. Cards with invalid parameters get a red error marker (hover it for the message).

Inspector (right). A form generated from the selected action's parameter schema — spin boxes for numbers, checkboxes for booleans, browse buttons for paths, plain text fields for comma-separated lists. Grey placeholder text shows values inherited from the YAML (currently the project's `facet_cutoffs`). Fields that only apply when a checkbox is on (like `cutoffs` under `Faceted`) are greyed out when it's off; their values are kept and restored when you re-enable the checkbox.

Preview + Save (bottom). A live YAML rendering of the current script — exactly what will be written to disk. **Save** writes *all* scripts back into the YAML the editor is open on — a `tracking_config.yaml`, or the relevant section of a `project.yaml` (everything else in the file is preserved); **Reload** discards unsaved changes and re-reads the file.

4. Building your first script

A typical everyday recipe — load, drop bad trackers, analyse, report:

1. Open the Script Editor, click **new**, and name the script (e.g. **full-run**).
2. Double-click **Load experiment** in the palette. Leave the path as **.** - it resolves to the Experiment Directory the script runs against.
3. Double-click **Filter trackers by quality**; in the Inspector set **min_high_quality** to **0.9**.
4. Double-click **Run Full Analysis**; leave *Faceted* checked and *cutoffs* blank to inherit the experiment's **facet_cutoffs**.
5. Double-click **Create PDF Report**.
6. Click **Save**.

The script now appears in the Hub's **Scripts** tile after that replicate is loaded. If the Hub already had it loaded, click **Reload** in the Scripts panel to re-read the file.

Rule of thumb: start standalone experiment scripts with **Load experiment.** Steps that need an experiment raise *"no experiment loaded — add a 'Load experiment' step first"* otherwise. The exceptions are scripts you always run from the Hub after loading an experiment, and scripts run through **run_in_experiments**; both receive a pre-loaded experiment.

5. Action reference

Load

Action	Key	Parameters	What it does
Load experiment	load_experiment	path (default .), force_preprocessing (default off)	Loads the experiment from an Experiment Directory (data/*.xlsx + tracking_config.yaml) and makes it the current experiment for all later steps. path of . (or blank) resolves to the script's own directory — the loaded experiment's dir, or the replicate being processed in a run_in_experiments run. force_preprocessing recomputes cached nearest-neighbour preprocessing.

QC / filtering

Action	Key	Parameters	What it does
Filter trackers by quality	filter_by_quality	min_high_quality (0–1, default 0.8)	Drops every tracker whose fraction of high-quality frames is below the threshold. Logs how many were kept.

Action	Key	Parameters	What it does
Filter trackers by region	<code>filter_by_region</code>	<code>regions</code> (comma-separated, e.g. <code>T_0, T_1</code>)	Keeps only trackers whose tracking region matches the list (prefix matching as a fallback). At least one region is required.
Run QC	<code>run_qc</code>	—	Prints the data-quality report and writes <code>{exp}_data_quality.csv</code> to <code>qc/</code> .

Filters change the in-memory experiment for the rest of the run — steps after a filter only see the surviving trackers. Order matters: filter *before* the analysis steps. (In the palette the two filters sit under **QC** and **Run QC** under **Analyze**; they are grouped together here because that is the order you use them in.)

Analyze

Action	Key	Parameters	What it does
Run Full Analysis	<code>run_analysis</code>	<code>facet</code> , <code>cutoffs</code>	The complete pipeline: experiment summary → QC → summary CSVs → statistics → plots. Mirrors the Hub's Run Analysis button.
Summarize	<code>summarize</code>	<code>facet</code> , <code>cutoffs</code>	Writes <code>{exp}_Summary.csv</code> and, when faceted, <code>{exp}_Summary_Facet.csv</code> .
Run pairwise comparisons	<code>run_pairwise_comparisons</code>	<code>facet</code> , <code>cutoffs</code>	Treatment-vs-treatment statistics for every metric relevant to the tracking type. Faceted runs write <code>{exp}_Stats.txt</code> ; flat runs write <code>{exp}_Stats_flat.txt</code> .
Create PDF Report	<code>create_report</code>	—	Builds <code>{exp}_report.pdf</code> with QC tables, tracker grids, and plots.

Plots

One action per plot type; the palette only shows those valid for the project's `tracking_type`. All take the same `facet` / `cutoffs` pair.

Action	Key	Available for
PI	<code>plot_pi</code>	<code>TWOCHOICETRACKER</code> , <code>TWOCHOICECOUNTER</code>
Percentage	<code>plot_percentage</code>	<code>TWOCHOICETRACKER</code> , <code>TWOCHOICECOUNTER</code>
Transitions	<code>plot_transitions</code>	<code>TWOCHOICETRACKER</code>
Total distance	<code>plot_totaldistance</code>	<code>TRACKER</code> , <code>TWOCHOICETRACKER</code> , <code>XCHOICETRACKER</code> ,

Action	Key	Available for
		PAIRWISEINTERACTIONTRACKER
Adjusted X position	plot_adjusted_x_position	XCHOICETRACKER
Interactions	plot_interactions	PAIRWISEINTERACTIONTRACKER, PAIRWISEINTERACTIONCOUNTER

When run from the Hub, every figure a plot step produces opens as a tab in the plot dock (respecting the **Interactive plots** toggle). Faceted variants get an "(facet)" suffix in the tab title.

6. Faceting: how **facet** and **cutoffs** work

Every analysis and plot action shares the same two parameters:

- **facet** (checkbox) — run the faceted variant, splitting the recording into time phases.
- **cutoffs** (comma-separated minutes, e.g. **10, 70**) — the phase boundaries. **10, 70** creates three phases: 0–10, 10–70, 70+ minutes.

Resolution order at run time:

1. **facet** off → flat (non-faceted) behaviour; **cutoffs** is ignored.
2. **facet** on and **cutoffs** filled in → those cutoffs are used.
3. **facet** on and **cutoffs** blank → the experiment's own **global.facet_cutoffs** from its YAML is used. (The Inspector shows this inherited value as grey placeholder text.)
4. **facet** on but no cutoffs anywhere → the step logs a notice and **falls back to flat** — it never fails just because cutoffs are missing.

Leaving **cutoffs** blank is the recommended default: across replicates each sub-experiment then uses *its own* configured cutoffs.

7. Running scripts

From the Hub (single project)

The **Scripts** card lists every script in the currently-selected YAML:

- **Run Script** runs the one selected in the drop-down.
- **Run All** runs every script in file order. The experiment state carries over from one script to the next (a script without a load step continues on whatever the previous script loaded).

Runs execute on a background thread — the log streams to the **Output** tab in real time as each step runs, failures (with tracebacks) also land in the **Errors** tab, and figures open as plot tabs.

If an experiment is already loaded in the Hub, a script that has *no* **load_experiment** step operates directly on it — handy for quick "filter-and-replot" recipes. A script *with* a load step always reloads fresh.

From a Project

The Hub's Project panel → **Analysis** card has its own **Script** picker and **Run script** button for Project Scripts — including the two built-ins, **Report pipeline** and **Standard pipeline**. A **Batch Run** runs a designated Project Script in every Project of a folder; see the user guide §8.5. See §8 below for the actions themselves.

Editing while the Hub is open

The Hub re-reads scripts when the Project is (re)loaded — reopening it from **Open Project** re-reads it from disk. After saving experiment scripts in the Script Editor, **Reload** in the Hub's **Scripts** panel picks up the changes without reloading anything else.

8. Project scripts (two-level scripting)

A **Project** (a directory of replicate experiments, see the user guide §3) has its own scripting level. Project Scripts live in `project.yaml` under `scripts:` and use **project actions**; they cannot contain experiment steps, and experiment scripts cannot contain project steps — the two levels meet in exactly one place, the `run_in_experiments` bridge.

Project actions

Action	Parameters	What it does
<code>validate_design</code>	—	Re-checks every replicate against the project design; fails the script on any mismatch. A cheap guard for the top of a pipeline.
<code>run_in_experiments</code>	<code>script</code> (name), <code>only</code> (replicate names; blank = all)	Runs a named Experiment Script in every replicate — or just the ones named in <code>only</code> (see below).
<code>render_publication_figures</code>	<code>format</code> (<code>svg</code> / <code>pdf</code> / <code>both</code> , default <code>svg</code>)	Writes the pooled publication figures to <code>figures/</code> from <code>plot_specs.yaml</code> — the Plot Editor's saves, headless. Skips itself when the project has no <code>plot_specs.yaml</code> .
<code>project_report</code>	<code>reports</code> (per-replicate reports, default on), <code>skip_analyzed</code> (default off)	The whole Create-report button in one step: analyzes every replicate (with its own report), pools the results into <code>analysis/</code> , then builds <code><project>_report.pdf</code> . Nothing needs to run before it. Leave <code>skip_analyzed</code> off to match the button, which always re-analyzes; note that a replicate whose removed regions have not reached its saved analysis is re-run even when it <i>is</i> on.
<code>generate_ai_narrative</code>	<code>provider</code> (<code>anthropic</code> /	Asks an AI provider to write the project narrative from the Combined Analysis. Soft-

Action	Parameters	What it does
	<code>openai</code>), <code>soft_fail</code> (default on)	fails by default — a provider error is logged and the script continues.

The `run_in_experiments` bridge

`run_in_experiments` takes one parameter — the **name** of an Experiment Script — and resolves it per replicate:

1. the Project's central `experiment_scripts`: section in `project.yaml` (author once, runs identically in every replicate), then
2. a script of that name in the replicate's own `tracking_config.yaml`.

Replicates run in order with a `[name]` prefix on every log line. A failing replicate does **not** stop the others; all failures are summarized when the script ends. Because each replicate is loaded fresh, the script does *not* need a `load_experiment` step.

The optional `only`: parameter narrows the run to a list of replicate directory names; blank means every replicate. The Inspector renders it as a checkable replicate list when the editor is open on a `project.yaml`. A name matching no replicate is logged and counted in the failure summary while the run continues — and the Hub pre-checks the script before running, so an unknown `only`: name (or a script name that resolves nowhere) aborts with a message before anything runs.

Migrating from the old batch mode: the retired *Batch experiments* mode ran a script named `batch` from each sub-folder's own config. That still works unchanged through the fallback: give the parent a `project.yaml` with the Hub's **Initialize existing directory...** button, then run a Project Script containing `run_in_experiments` with `script: batch`.

The two built-in pipelines

The Analysis card's script picker always offers both built-ins. Neither is ever written to your yaml, so both always match the shipped default.

Standard pipeline

1. `validate_design`
2. `project_report`
3. `render_publication_figures` (SVG)

Report pipeline

1. `project_report`
2. `render_publication_figures` (SVG) — dropped for a Project that has no `plot_specs.yaml`, so an unattended run never invents uncurated default-spec figures

The Report pipeline is the **Create report** button plus curated figures, and it is what a **Batch Run** executes in every Project unless another script is designated (user guide §8.5). It omits the `validate_design` gate on purpose: that would fail Projects mid-migration and stop an overnight run. Its steps are also what every new `project.yaml` is seeded with, under the script name `batch`.

In both, the figure step comes *after* the report: `project_report` is what analyzes the replicates, and `render_publication_figures` reads their saved summaries.

Zero authoring gets a complete project run; your own scripts appear alongside the built-ins in the picker.

Retired actions. `run_all_analyses` and `build_combined_analysis` used to be separate steps. A script action mirrors an Analysis-card button, and there is no Run-all or Build-combined button — both are part of what **Create report** does — so both actions were folded into `project_report`. Scripts you saved before that still run: the retired steps are absorbed automatically (and `project_report` takes the position of the first one, so a later figure step still finds analyzed replicates). The Script Editor flags them so you can delete them.

Central experiment scripts

`project.yaml`'s `experiment_scripts`: holds experiment-LEVEL scripts at the project, in the same spirit as the `design`: section: one definition, every replicate. Edit them in the Script Editor's *Experiment scripts* level — the palette and validation are exactly the experiment ones (filtered by the design's experiment type).

```
# project.yaml
design: { ... }
experiment_scripts:
- name: nightly
  steps:
  - action: run_analysis
    params: {}
  - action: create_report
    params: {}
scripts:
- name: full-pipeline
  steps:
  - action: validate_design
    params: {}
  - action: run_in_experiments
    params: {script: nightly}
  - action: project_report
    params: {}
  - action: render_publication_figures
    params: {format: both}
```

9. The YAML behind it all

The Script Editor is a front-end for a plain YAML structure you can also read and edit by hand. Scripts live under the top-level `scripts`: key of `tracking_config.yaml`:

```
scripts:
- name: nightly
  steps:
```

```

- action: load_experiment
  params:
    path: '.'
    force_preprocessing: false
- action: filter_by_quality
  params:
    min_high_quality: 0.9
- action: run_analysis
  params:
    facet: true
    cutoffs: ''
- action: create_report
  params: {}
- name: quick-pi                                # ← a second, ordinary script
  steps:
    - action: load_experiment
      params: {path: '.'}
    - action: plot_pi
      params: {facet: true, cutoffs: '10, 70'}

```

Hand-editing rules:

- **scripts:** is a **list**; each entry needs **name** (string) and **steps** (list). A missing name displays as "Untitled".
- Each step needs **action** (one of the keys in §5) and **params** (a mapping; **{}** or omitted is fine for actions without parameters).
- Unknown **action** keys and malformed parameter values are caught by validation before anything runs (§10).
- Parameters you omit take their defaults; parameters not in the action's schema are ignored.
- Saving from the Script Editor rewrites only the **scripts:** key — the rest of the YAML (global, regions) is preserved. Deleting the last script removes the **scripts:** key entirely.

10. Validation and troubleshooting

Scripts are validated at two points:

- **In the editor**, continuously: step cards with problems get a red marker, and the Inspector shows the messages inline. Some checks (anything depending on the tracking type) can only run once an experiment is loaded, so they are deferred to run time.
- **At run time**, before any step executes: the whole script is checked (unknown actions, missing required parameters, malformed **cutoffs**, ...). If anything fails, the run aborts with a list like **Step 2: cutoffs must be a comma-separated list of integers** and **no steps run at all** — a script never executes halfway on a validation error.

Common messages and their fixes:

Message	Fix
Script has no steps	The script is empty — add steps, or delete it.

Message	Fix
<code><action>: no experiment loaded — add a 'Load experiment' step first.</code>	Put <code>Load experiment</code> at the top. (Not needed for scripts run via <code>run_in_experiments</code> — each replicate is pre-loaded.)
<code>cutoffs must be a comma-separated list of integers</code>	Use e.g. <code>10, 70</code> — minutes, integers only.
<code>unknown action '<key>'</code>	Hand-edited YAML has a typo in <code>action:</code> — compare with the keys in §5.
<code>run_in_experiments: <rep>: no script named '...'</code>	Define the script in <code>project.yaml</code> <code>experiment_scripts:</code> (preferred) or in that replicate's own <code>tracking_config.yaml</code> .
Script ran but faceted output is missing	No cutoffs anywhere (step blank + no <code>facet_cutoffs</code> in the YAML) — the step fell back to flat. Add cutoffs in either place.

Where to look when something goes wrong: the **Output** tab holds the full chronological log of the run (each step logs a — `Step i/N: ...` header), and the **Errors** tab collects just the failures and warnings, with tracebacks, so nothing scrolls out of sight.